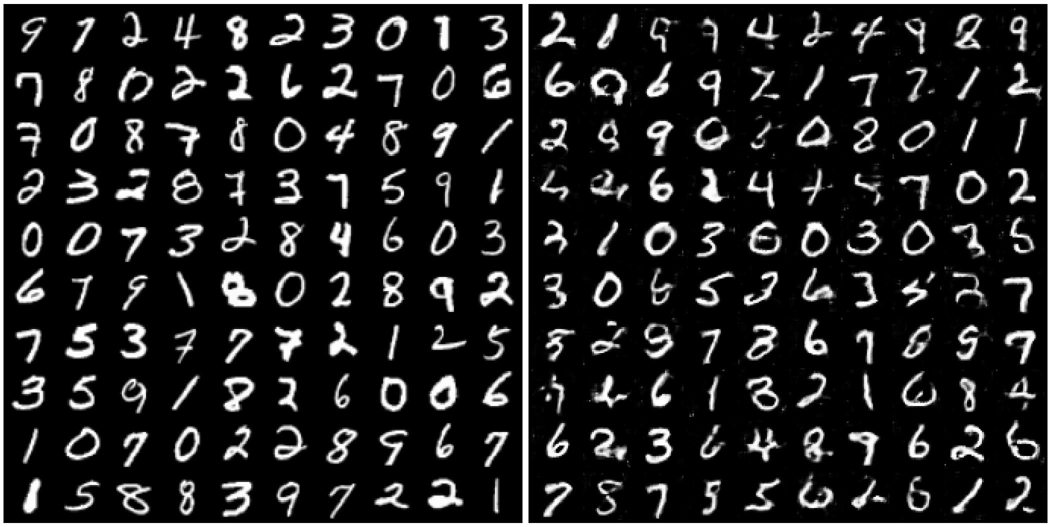
B.2. Dataset visualization

Figure 4 shows images from real-*MNIST* and "fake"-*MNIST*, while Figure 5 shows samples from *CIFAR-10* and *CIFAR-10.1*.



(a) Real-*MNIST* (b) "Fake"-*MNIST*

Figure 4. Images from real-*MNIST* and "fake"-*MNIST*. "Fake"-*MNIST* is generated by DCGAN (Radford et al., 2016).

B.3. Configurations

We implement all methods on Python 3.7 (Pytorch 1.1) with a NIVIDIA Titan V GPU. We run ME and SCF using the official code (Jitkrittum et al., 2016), and implement C2ST-S, C2ST-L, MMD-D and MMD-O by ourselves. We use permutation test to compute p-values of C2ST-S and C2ST-L, MMD-D, MMD-O and tests in Table 4. We set $\alpha = 0.05$ for all experiments. Following Lopez-Paz & Oquab (2017), we use a deep neural network F as the classifier in C2ST-S and C2ST-L, and train the F by minimizing cross entropy. To fairly compare MMD-D with C2ST-S and C2ST-L, the network ϕ_ω in MMD-D has the same architecture with feature extractor in F. Namely, $F = g \circ \phi_\omega$, where g is a two-layer fully-connected network. The network g is a simple binary classifier that takes extracted features (through ϕ_ω) as input. For test methods shown in Table 4, the network ϕ_ω in them also has the same architecture with that in MMD-D.

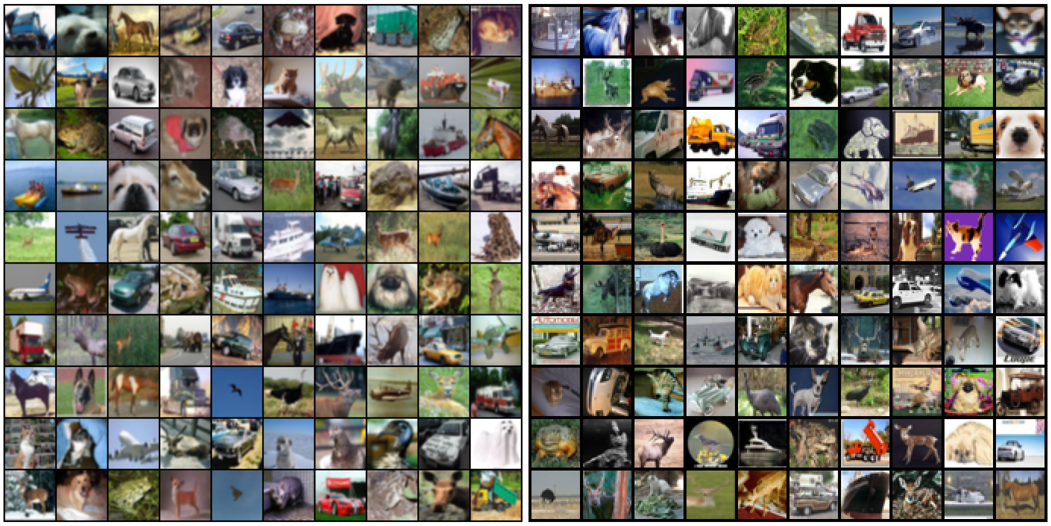
For *Blob*, *HDGM* and *Higgs*, ϕ_ω is a five-layer fully-connected neural network. The number of neurons in hidden and output layers of ϕ_ω are set to 50 for *Blob*, $3 \times d$ for *HDGM* and 20 for *Higgs*, where d is the dimension of samples. These neurons are with softplus activation function, i.e., $\log(1 + \exp(x))$. For *MNIST* and *CIFAR*, ϕ_ω is a *convolutional neural network* (CNN) that contains four convolutional layers and one fully-connected layer. The structure of the CNN follows the structure of the feature extractor in the discriminator of DCGAN (Radford et al., 2016) (see Figures 6 and 8 for the structure of ϕ_ω in MMD-D, and Figures 7 and 9 for the structure of classifier F in C2ST-S and C2ST-L). The link of DCGAN code is `https://github.com/eriklindernoren/PyTorch-GAN/blob/master/implementations/dcgan/dcgan.py`.

We use Adam optimizer (Kingma & Ba, 2015) to optimize 1) parameters of F in C2ST-S and C2ST-L, 2) parameters of ϕ_ω in MMD-D and 3) kernel lengthscale in MMD-O. We set drop-out rate to zero when training C2ST-S, C2ST-L and MMD-D on all datasets.

B.4. Detailed parameters of all test methods

In this subsection, we demonstrate detailed parameters of all test methods. Except for learning rate of Adam optimizer, we use default parameters of Adam optimizer provided by Pytorch. We use one validation set (with the same size of training set) to roughly search these parameters. Using these parameters, we compute test power of each test method on 100 test sets (with the same size of training set).

For ME and SCF, we follow Chwialkowski et al. (2015) and set $J = 10$ for *Higgs*. For other datasets, we set $J = 5$.



(a) *CIFAR-10* test set (b) *CIFAR-10.1* test set

Figure 5. Images from *CIFAR-10* test set and the new *CIFAR-10.1* test set (Recht et al., 2019).

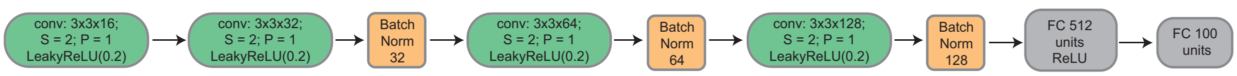


Figure 6. The structure of ϕ_ω in MMD-D on *MNIST*. The kernel size of each convolutional layer is 3; stride (S) is set to 2; padding (P) is set to 1. We do not use dropout. Best viewed zoomed in.

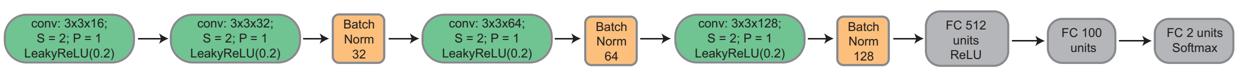


Figure 7. The structure of classifier F in C2ST-S and C2ST-L on *MNIST*. The kernel size of each convolutional layer is 3; stride (S) is set to 2; padding (P) is set to 1. We do not use dropout. In the first layer, we will convert the *CIFAR* images from $32 \times 32 \times 3$ to $64 \times 64 \times 3$. Best viewed zoomed in.

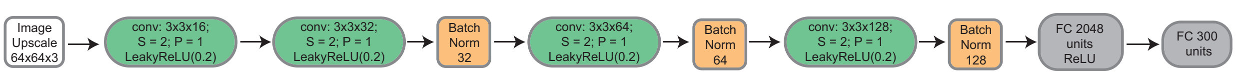


Figure 8. The structure of ϕ_ω in MMD-D on *CIFAR*. The kernel size of each convolutional layer is 3; stride (S) is set to 2; padding (P) is set to 1. We do not use dropout in all layers. In the first layer, we will convert the *CIFAR* images from $32 \times 32 \times 3$ to $64 \times 64 \times 3$. Best viewed zoomed in.

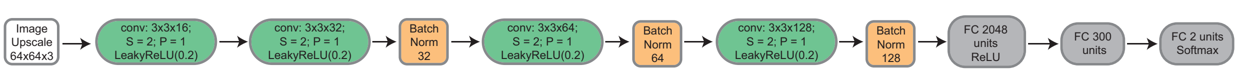


Figure 9. The structure of classifier F in C2ST-S and C2ST-L on *CIFAR*. The kernel size of each convolutional layer is 3; stride (S) is set to 2; padding (P) is set to 1. We do not use dropout. Best viewed zoomed in.

For C2ST-S and C2ST-L, we set batchsize to $\min\{2 \times n_b, 128\}$ for *Blob*, 128 for *HDGM* and *Higgs*, and 100 for *MNIST* and *CIFAR*. We set the number of epochs to $500 \times 18 \times n_b/$batchsize for *Blob*, $1,000$ for *HDGM*, *Higgs* and *CIFAR*, and $2,000$ for *MNIST*. We set learning rate to 0.001 for *Blob*, *HDGM* and *Higgs*, and 0.0002 for *MNIST* and *CIFAR* (following Radford et al. (2016)).

For MMD-O, we use full batch (i.e., all samples) to train MMD-O. we set the number of epochs to $1,000$ for *Blob*, *HDGM*, *Higgs* and *CIFAR*, and $2,000$ for *MNIST*. We set learning rate to 0.0005 for *Blob*, *MNIST* and *CIFAR*, and 0.001 for *HDGM*.

Table 7. Results on *Higgs* ($\alpha = 0.05$). We report average Type I error on *Higgs* dataset when increasing number of samples (N). Note that, in *Higgs*, we have two types of Type I errors: 1) Type I error when two samples drawn from $\mathbb{P}$ (no Higgs bosons) and 2) Type I error when two samples drawn from $\mathbb{Q}$ (having Higgs bosons). Type I reported here is the average value of 1) and 2). Since Type I error reported here is the average value of two average Type I errors, we do not report standard errors of the average Type I error in this table.

N	ME	SCF	C2ST-S	C2ST-L	MMD-O	MMD-D
1000	0.048	0.040	0.043	0.048	0.059	0.037
2000	0.043	0.032	0.060	0.056	0.055	0.053
3000	0.049	0.043	0.046	0.053	0.051	0.069
5000	0.056	0.035	0.052	0.065	0.049	0.062
8000	0.050	0.034	0.065	0.067	0.056	0.037
10000	0.059	0.032	0.057	0.058	0.045	0.048
Avg.	0.051	0.036	0.054	0.058	0.050	0.051

Table 8. Results on *MNIST* given $\alpha = 0.05$. We report average Type I error±standard errors on real-*MNIST* vs. real-*MNIST* when increasing number of samples (N).

N	ME	SCF	C2ST-S	C2ST-L	MMD-O	MMD-D
200	0.076±0.011	0.075±0.010	0.035±0.006	0.045±0.005	0.068±0.004	0.056±0.003
400	0.062±0.010	0.056±0.007	0.044±0.006	0.040±0.004	0.053±0.005	0.056±0.005
600	0.051±0.003	0.049±0.009	0.039±0.005	0.054±0.007	0.066±0.008	0.056±0.008
800	0.054±0.006	0.046±0.006	0.043±0.005	0.042±0.007	0.051±0.005	0.054±0.007
1000	0.047±0.006	0.045±0.010	0.038±0.006	0.046±0.005	0.041±0.007	0.062±0.006
Avg.	0.058	0.054	0.040	0.045	0.056	0.057

For MMD-D, we use full batch (i.e., all samples) to train MMD-D with samples from *Blob*, *HDGM* and *Higgs*. We use mini-batch (batchsize is 100) to train MMD-D with samples from *MNIST* and *CIFAR*. We set the number of epochs to $1,000$ for *Blob*, *HDGM*, *Higgs* and *CIFAR*, and $2,000$ for *MNIST*. We set learning rate to 0.0005 for *Blob* and *Higgs*, 10^{-5} for *HDGM*, 0.001 for *MNIST* and 0.0002 for and *CIFAR* (following Radford et al. (2016)).

B.5. Links to datasets

Higgs dataset can be downloaded from UCI Machine Learning Repository. The link is `https://archive.ics.uci.edu/ml/datasets/HIGGS`.

MNIST dataset can be downloaded via Pytorch. See the code in `https://github.com/eriklindernoren/PyTorch-GAN/blob/master/implementations/dcgan/dcgan.py`.

CIFAR-10.1 is available from `https://github.com/modestyachts/CIFAR-10.1/tree/master/datasets` (we use `cifar10.1_v4_data.npy`). This new test set contains $2,031$ images from TinyImages (Torralba et al., 2008).

B.6. Type I errors on *Higgs* and *MNIST*

Table 7 shows average Type I error on *Higgs* dataset when increasing number of samples (N). Table 8 shows average Type I error on real-*MNIST* vs. real-*MNIST* when increasing number of samples (N).

C. Interpretability on *CIFAR-10* vs *CIFAR-10.1*

In Section 7.1, we have shown that images in *CIFAR-10* and *CIFAR-10.1* are not from the same distribution. Thus, it is interesting to try to understand the major difference between the datasets. Mean Embedding tests (Chwialkowski et al., 2015) compare the mean embeddings $\mu_{\mathbb{P}}$ and $\mu_{\mathbb{Q}}$ at test locations $v_1, \dots, v_L$, rather than through their overall norm. The test statistic is

$$\hat{\Lambda} = n\bar{z}_n^{\mathsf{T}} S^{-1}\bar{z}_n, \quad z_i = (k(x_i, v_j) - k(y_i, v_j))_{j=1}^{L} \in \mathbb{R}^L, \quad \bar{z}_n = \frac{1}{n}\sum_{i=1}^{n} z_i, \quad S_n = \frac{1}{n-1}\sum_{i=1}^{n}(z_i - \bar{z}_n)(z_i - \bar{z}_n)^{\mathsf{T}};$$